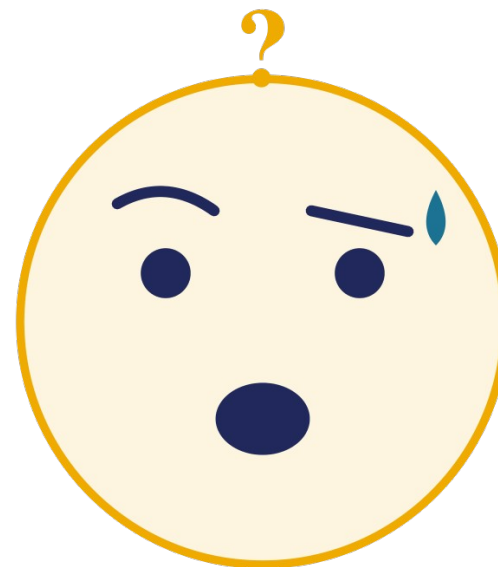


temperature=0 — значит, ответ всегда одинаковый?



T=0

Да.



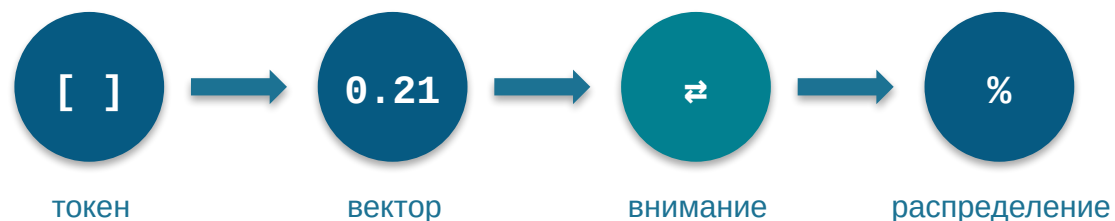
→ **Но нет.**

почему — сегодня

Как работают современные большие модели

02

Конвейер инференса — и границы, которые меняют инженерные решения



Карта лекции — 7 разделов

0 **Введение** — *рамка и конвейер целиком*

1 **Токенизация** — *как модель видит ваш текст*

2 **Эмбеддинги** — *пространство смыслов и граница похожести*

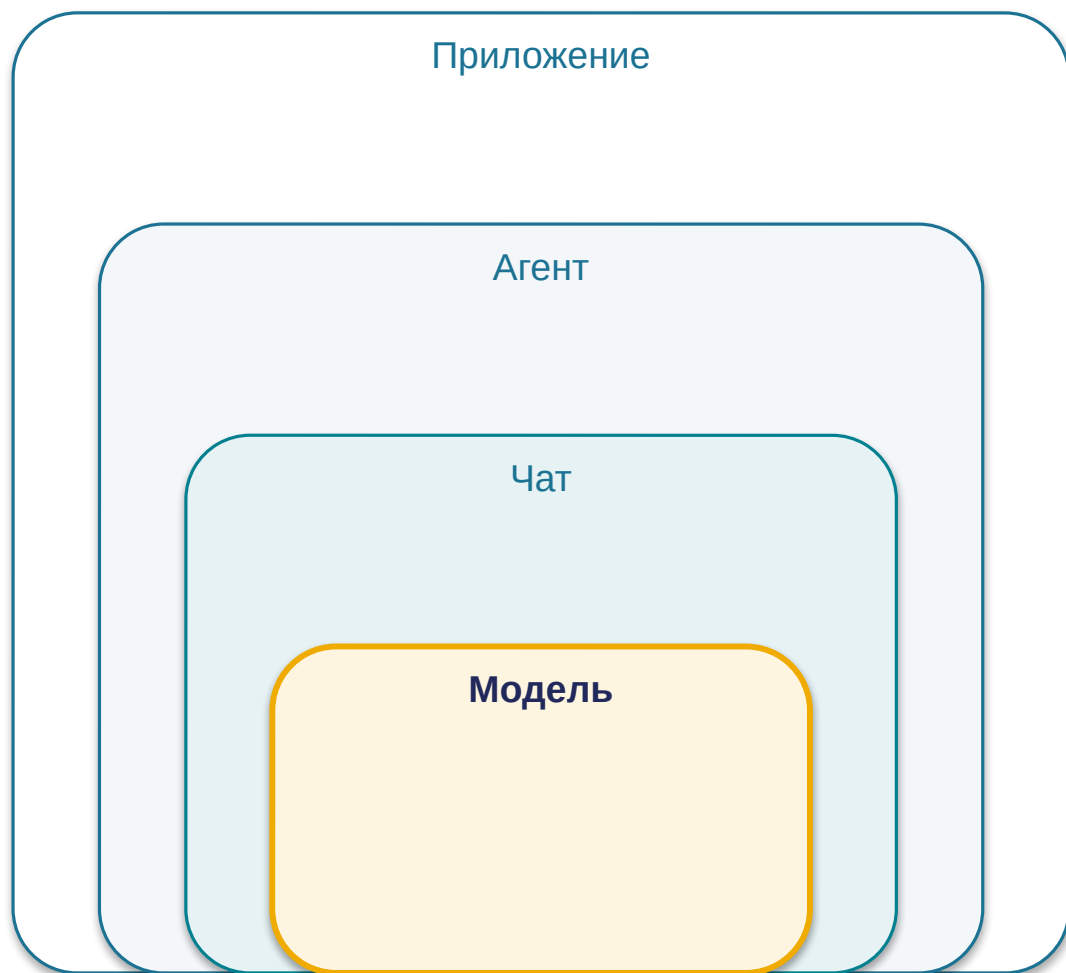
3 **Механизм внимания** — *что важно сейчас: роли, кэш, длинный контекст*

4 **Сэмплинг** — *от распределения к токену: температура, детерминизм, невидимые токены*

5 **Виды и размеры моделей** — *на чём модели запускаются, мультимодальность, ландшафт 2026*

6 **Финал** — *сборка конвейера, итоги механизмов*

Объект сегодня — слой «модель» из четырёх слоёв Лекции 1



Слой «модель»: stateless-инференс — на вход данные, на выход предсказание, без памяти между вызовами.

Сегодня: разбираем, что происходит внутри этого инференса — и где его устройство меняет инженерные решения.

Цель лекции

«Рассмотреть, как работает языковая модель, — и разобраться с **важными деталями**, которые меняют то, как вы строите промпты, агентов и решения.»

почему исправленный strawberry ничего не доказывает

почему промпт с ролью реально меняет ответ

что на деле умеет окно 1M

почему $T=0$ не даёт одинаковых ответов

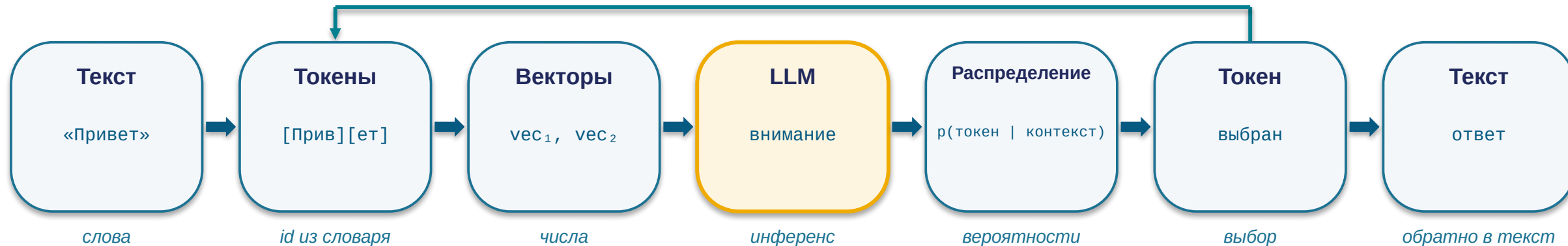
сколько стоят невидимые токены рассуждения

маленькая модель или гигант — по какому критерию выбирать

чем заменить веру в бенчмарки

Поток данных в LLM — туда и обратно

↻ токены генерируются по одному; каждый добавляется ко входу



Раздел 1

Текст → Токены

Раздел 2

Токены → Векторы

Раздел 3

LLM: внимание

Раздел 4

Распределение → Токен

Слова — только на границах; внутри — векторы.



Раздел 1

Токенизация

«Как модель видит ваш текст»

3 разбора · 3 провала



Токен — id из словаря модели: не буква и не слово

cat → [cat] → 1 токен

tokenization → [token][ization] → 2 токена

клубника → [к][луб][ника] → 3 токена (o200k_base)

«В среднем: 1 токен $\approx$ 4 символа EN $\approx$ 2 символа RU»

Словарь и модель — два разных артефакта: словарь строится до обучения модели, отдельным алгоритмом на своём корпусе.

ВРЕ — компромисс между алфавитом и словарём

Не все буквы (долго) и не все слова (незнакомое выпадает) — частые подпоследовательности.

Обучающий корпус

low
lower
newest
widest



ВРЕ-словарь

low · er · new · est · wid

+ одиночные символы, частые слоги, целые слова

Словарь строится один раз до обучения; на инференсе — выборка готовых правил, не вычисление.

Разные вендоры режут один и тот же текст по-разному: Claude, GPT, Gemini — свои словари и правила.

Гонка патчей: strawberry починили в апреле, cranberry — только в июле

Механизм — слепота к буквам

strawberry →

[st][raw][berry]

Модель видит **3 токена**, не 10 букв.

Каждый вирусный кейс патчится точно; класс задач остаётся проблемным.

GPT-5.2 · дек 2025

strawberry ✗ — «в strawberry две r»

GPT-5.5 · апр 2026

strawberry ✓ / cranberry ✗ — «две r» вместо трёх

GPT-5.6 · июль 2026

cranberry ✓ — очередной вирусный кейс запатчен

StrawberryBench

847 вопросов, 7 уровней сложности — системная проверка вместо вирусного вопроса

«Рванный интеллект» (jagged intelligence): уровень навыка задаётся данными обучения, а не «общим умом» — модель берёт олимпиадное золото и проваливает подсчёт букв.

Что делать: тестируйте «cranberry своей предметки» — редкие случаи, на которых никто не хайпил; вирусное прохождение ≠ навык.

Токенизатор режет по частоте, а не по структуре

Числа и код — самые частые «нетекстовые» входы: от их нарезки зависят арифметика модели и ваш бюджет токенов.

Числа

10000000 → [100][000][0]

Чанки слева направо — а математические разряды считаются справа налево: границы не совпадают, сложение в столбик ломается.

Нарезка справа налево улучшает арифметику; задача-специфичные схемы — **до +33% точности** к стандартной нарезке.

Код

GPT-2: 16 токенов на отступ 4-го уровня.

GPT-4: пробелы группами — словарь чинится, но не под все задачи сразу.

Что делать:

Разделители разрядов («1 234 567»)

Вычисления — в инструмент

Единообразные отступы

Готовые чат-продукты уже сами уводят счёт во встроенные инструменты (code interpreter); звать инструмент самому — нестандартные случаи и свои приложения поверх API.

Порядка 4% словаря — glitch-токены

SolidGoldMagikarp (2023)

Юзернейм с Reddit в словаре GPT — модель не могла его повторить.

Механизм: корпус словаря $\neq$ корпус модели $\rightarrow$ эмбединг токена остаётся у случайной инициализации — вектор «ничего не значит».

На что влияет

- Сбои парсинга экзотических строк — редкие идентификаторы, обфусцированный текст, необычный Unicode
- Риски в проде — логи, user-generated ID, произвольный пользовательский ввод
- Необъяснимые ответы «невпопад» без ошибки в коде

GlitchMiner (AAAI 2026)

порядка 4% словаря по одной из оценок;

воспроизводится в открытых семействах Llama, Qwen, Gemma, Phi-3, Mistral.

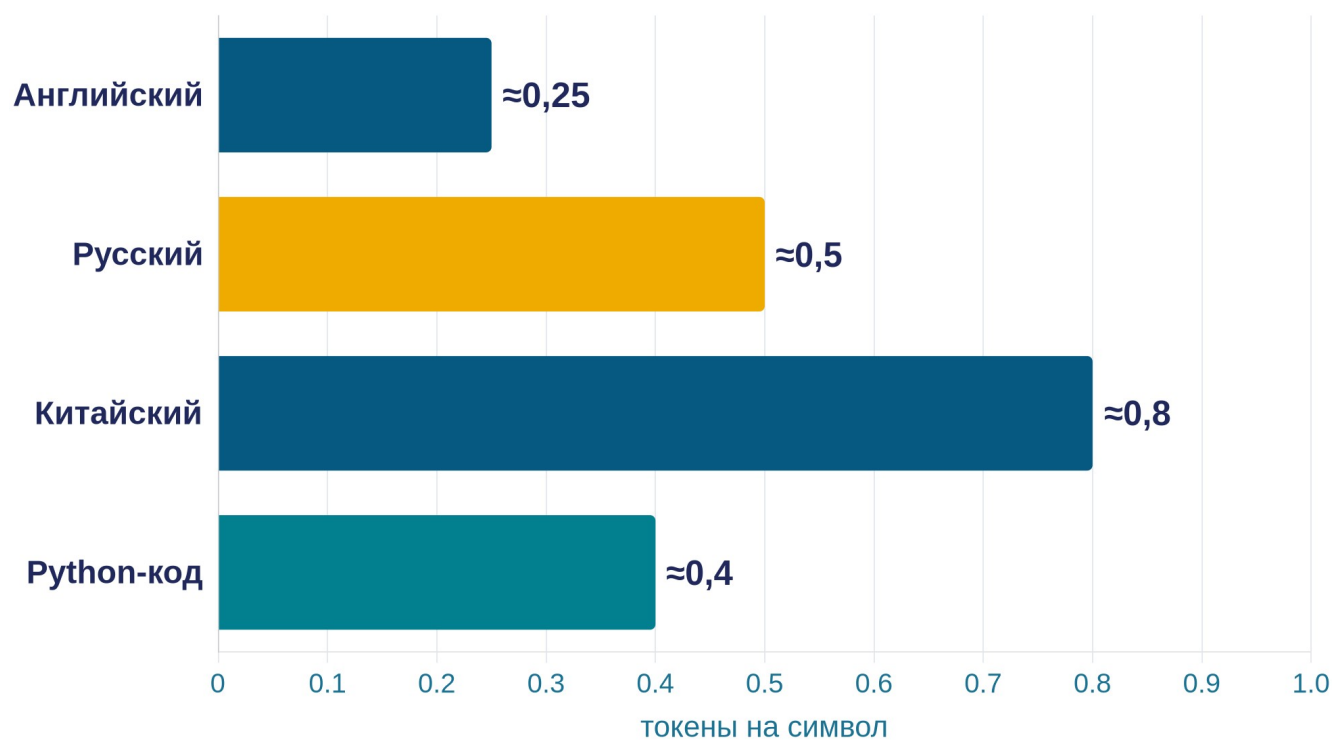
Что на практике:

Необъяснимое поведение на экзотических строках (редкие идентификаторы, обфусцированный текст, необычный Unicode)?
Гипотеза: glitch-токен. Проверка: замените строку плейсхолдером.

В продуктах, принимающих произвольный ввод, — нормализация и санитизация входа до модели.

Системная особенность конвейера, не баг версии — масштабом не чинится.

Русский текст стоит $\approx 2\times$ дороже английского



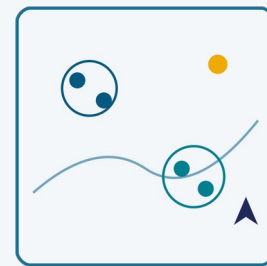
словари GPT-семейства

Переход на o200k_base:

примерно **-35%** нелатинским языкам — разрыв сокращается, но не исчезает.

Что делать:

- Калибруйте лимиты в токенах на своём языке: фрагменты для поиска, `max_tokens`, бюджет окна.
- Пакетная обработка больших объёмов — оцените перевод на английский ($\approx 2\times$ дешевле); в интерактиве разница того не стоит.



Раздел 2

Эмбе́ддинги

«Пространство смыслов — и граница похожести»

4 разбора · 1 провал



Каждому токenu — вектор из входной таблицы модели



Размерности

text-embedding-3-small — 1536

text-embedding-3-large — 3072

внутренние размерности фламанов не публикуются; порядок — тысячи

«Геометрическая близость = смысловая близость»

Что делать: опечатка или другой регистр — уже другой токен и другой вектор; нормализуйте вход до эмбединга.

Эмбе́ддинги — не только внутренность инференса, но и инструмент поиска

Когда вы строите поиск или RAG, вы пользуетесь третьей жизнью термина «эмбе́ддинг» — отдельной *embedding*-моделью.

1

Входная таблица

внутри инференса

Статическая: вектор [кот] один и тот же в любом предложении. Выборка по id, контекста ещё нет.

2

Представление данных внутри модели

внутри инференса

Векторы, которые модель пересчитывает по ходу чтения контекста — после слоёв внимания. Именно они несут «понимание» модели.

3

Векторы для поиска

самостоятельный инструмент

Вектор целого текста от отдельной *embedding*-модели — не внутренности вашего чат-LLM. Свой продукт, своё обучение, свои лидерборды.

это и есть ваш поиск/RAG

Обновили чат-LLM → переиндексировать базу НЕ надо: индекс живёт в координатах *embedding*-модели.

Близкие по смыслу токены лежат рядом — в сотнях-тысячах измерений

2D-проекция (PCA-стиль) каждая из 1536+ осей — выученный признак; здесь показаны две

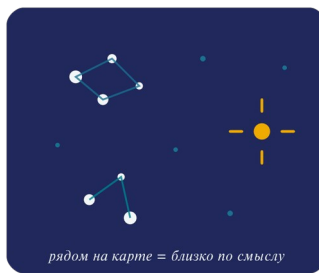
ось $\approx$ признак: инфраструктура $\leftrightarrow$ фронтенд

Как настроить SSL

Установка
HTTPS-сертификата

Деплой React-компонента

Сборка
React-приложения



рядом на карте = близко по смыслу



Рецепт борща

выброс — другая область

ось $\approx$ признак: тематика (веб-разработка $\leftrightarrow$ кулинария)

Размерность

Публичные embedding-модели: **1536–3072** измерения; внутренние у флагманов — порядка тысяч.

Обучение

Координаты не задаются вручную: похожие контексты употребления $\rightarrow$ близкие векторы.

Проекция

Увидеть пространство можно только через PCA/t-SNE — 2D-картинка теряет часть структуры.

Что делать: близость измерима расстоянием — фильтрация и кластеризация без разметки и без LLM возможны прямо на векторах, дёшево.

Высокое сходство — «об одном и том же», не «с одинаковым смыслом»

cosine similarity	SSL	HTTPS	React-к.	React-п.	Борщ
Как настроить SSL	1,00	0,85	0,18	0,20	0,08
Установка HTTPS-сертификата	0,85	1,00	0,22	0,19	0,07
Деплой React-компонента	0,18	0,22	1,00	0,78	0,12
Сборка React-приложения	0,20	0,19	0,78	1,00	0,10
Рецепт борща	0,08	0,07	0,12	0,10	1,00

«Как настроить SSL» ↔

«Как отключить SSL»

Очень высокое сходство — противоположный практический смысл.

шкала: 0 — светлое · 1 — тёмное

Similarity — сигнал кандидатов; релевантность — отдельная задача: реранкер, гибрид, фильтры.

Эмбединги — фундамент понимания: модель работает с векторами, не строками



Перефразирования и синонимы

«Как настроить SSL» и «Установка HTTPS-сертификата» — близкие векторы → модель отвечает одинаково; то же с «авто» и «машина».

Межъязыковая близость

«клубника» и strawberry — близкие векторы → ответ корректен независимо от языка запроса.

Семантическая близость на уровне предложений — основа «понимания» переформулировок.

Что делать: одна embedding-модель и индекс обслуживают поиск, кластеризацию и RAG сразу — смена модели означает переиндексацию всего хранилища; выбирайте как инфраструктурное решение.

Выбор embedding-модели под задачу (МТЕВ, матричные представления) — материал для самостоятельного чтения.

Раздел 3



Механизм внимания

«Как модель решает, на что опереться в контексте — и что из этого следует для ролей, кэша и длинных окон»

4 разбора · 2 провала



Внимание — это сверка каждого токена со всеми остальными

Каждый токен «смотрит» на все остальные одновременно. На каждом шаге — $N \times N$ связей.

вес	Кот	съел	мышь	потому что	он	был	голоден
Кот	1,0	0,3	0,2	0,1	0,1	0,1	0,0
съел	0,4	1,0	0,5	0,1	0,1	0,1	0,1
мышь	0,2	0,4	1,0	0,1	0,1	0,0	0,1
потому что	0,1	0,2	0,2	1,0	0,3	0,2	0,2
он	0,7	0,1	0,2	0,2	1,0	0,3	0,4
был	0,1	0,2	0,2	0,3	0,4	1,0	0,5
голоден	0,1	0,2	0,3	0,3	0,4	0,5	1,0

По строке «он»: наибольший вес — на «Кот» (мужской род). Статистическая связь, выученная на корпусе.

В декодере токен видит только предыдущие — показана полная сверка для наглядности.

Размерность

$N \times N$, где N — длина контекста. Удвоение контекста — **учетверение вычислений внимания**.

На каждом шаге

Распределение весов пересчитывается заново на каждом шаге генерации.

Многоголовость

В каждом слое — десятки параллельных «голов» (типично 32–128); каждая ловит свой тип связей: грамматика, семантика, дальние зависимости.

На что влияют веса: они определяют, чьи Value попадут в представление текущего токена — и напрямую формируют следующее предсказание.

Внимание — распределение весов на весь контекст: три проекции Query / Key / Value

Q — про текущий шаг. K и V — про уже обработанный контекст.

Query — «что я сейчас ищу»

Key — «что я предлагаю»

Value — «что я отдам, если меня выбрали»

«Кот съел мышь, потому что **он** был голоден»

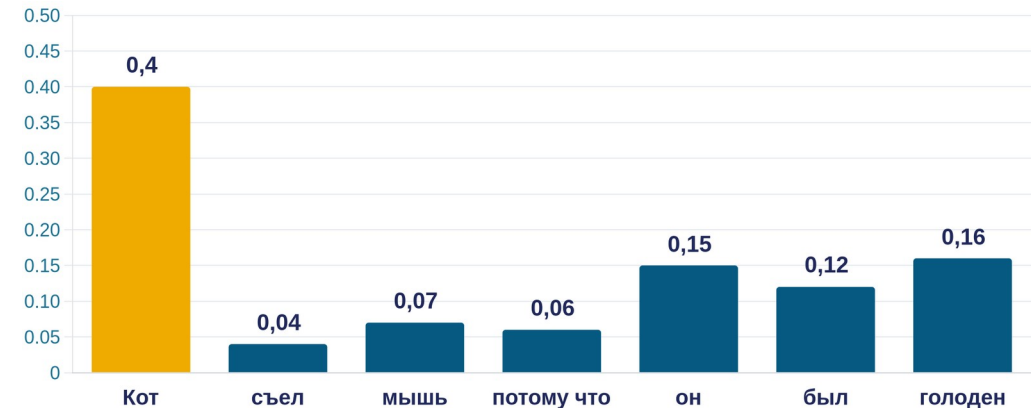
Q («он») = «ищу: кто мог быть голоден»

K («Кот») = «я — одушевлённый субъект»

V («Кот») = содержимое, которое вольётся в представление «он»

Совпадение Q («он») и K («Кот») даёт высокий вес → V («Кот») определяет обновлённое представление токена «он».

Распределение весов — сумма = 1, лидер «Кот»



1. На вход — все токены контекста.

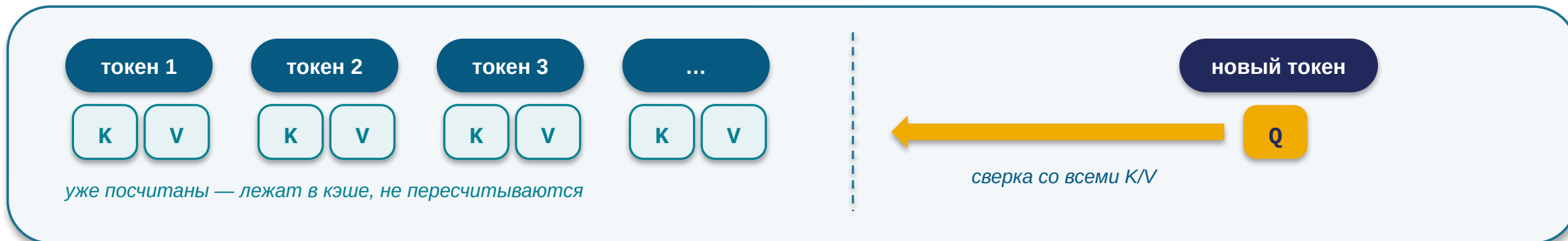
2. На выходе — распределение весов, сумма = 1.

3. Пересчитывается на каждом шаге генерации.

Метафора: фонарик в тёмной комнате — луч направлен на релевантные токены, яркость света = вес внимания.

К и V кешируются — заново считается только Q

KV-cache: Key/Value обработанных токенов сохраняются в памяти ускорителя. На каждом шаге вычисляется только Q нового токена — против сохранённых K/V.



Фаза 1 — prefill (обработка промпта)

- Все токены входа известны сразу → K/V считаются **параллельно**
- Упирается в **вычислительную мощность**
- Определяет **паузу до первого символа ответа (TTFT)**



Фаза 2 — decode (генерация ответа)

- Строго **последовательно**, токен за токеном
- Каждый шаг читает **весь накопленный кэш** из памяти
- Упирается в **пропускную способность памяти** → скорость «печати»

Кэш у провайдеров: OpenAI — авто · Gemini — implicit (авто) · DeepSeek — авто-дисковый · Anthropic — явный (cache_control)

KV-cache делает повторную подачу истории дешёвой и быстрой; «тормозит и дорожает» — когда кэш НЕ срабатывает (длинный чат без кэша, нестабильный префикс).

Кэш промптов — ставка на повторное использование, а не скидка

Три запроса подряд:

Запрос 1: префикс пишется в кэш — запись 1.25–2× ставки

Запрос 2: тот же префикс → **cache hit** — чтение 0.1× (новейшие — 0.025×)

Запрос 3: строка добавлена в начало → префикс не совпал → мимо кэша, полная цена

Кейс: 50 000 анализов документов в месяц

без кэша

\$45 000

\$8 000 с кэшем · **−82%**

Условие: точное совпадение префикса. Один изменившийся токен инвалидирует кэш для всего, что после него.

системный промпт

✓ кэш

инструкции

✓ кэш

документы

✓ кэш

«Сегодня 2026-09-05 14:23» — динамический

!

вопрос

× мимо кэша

Динамический элемент в начале (timestamp) рушит кэш всех блоков после него — это и есть «запрос 3».

Правило компоновки: стабильное — в начало (промпт, инструкции, примеры, документы), переменное — в конец.

Роль работает через вес во внимании — но не повышает фактическое качество

Рабочий пример: куда смотрит «он»

«Кот съел мышь, потому что он был голоден»

←
главный вес

→
Подумайте 30 секунд: куда уйдёт вес от «он» в «Программа упала, потому что он забыл обработать null»?

Упрощение: агрегат сотен связей в десятках слоёв — модель воспроизводит корреляции употребления, не грамматический разбор.

Без роли

«Объясни GIL» → нейтральный, обобщённый ответ.



С ролью

«Ты — опытный Python-разработчик. Объясни GIL» — токены роли получают вес → смещают распределение следующих токенов: конкретнее, в экспертном регистре.

Систематические проверки показывают: роль меняет тон, стиль и отбор содержания — но не повышает фактическое качество ответов.

Роль — инструмент управления стилем и фокусом, а не «усилитель ума». Нужен ответ по вашим данным — дайте данные, а не третье прилагательное к слову «эксперт».

Фронтир-стандарт 2026 — окно до 1 млн токенов. Но заявленное ≠ полезное

GPT-3.5 (2022) — 4 тыс.



Claude 3.5 (2024) — 200 тыс.



Фронтир-стандарт (2026) — **до 1 млн** · Fable 5, GPT-5.6, Gemini 3.1 Pro и др.



Gemini 3.5 Pro (2026) — 2 млн · единичная модель, не стандарт



ширина — логарифмическая шкала

Llama 4 Scout: «10 млн»

заявлено; ни один опубликованный бенчмарк не подтверждает качество вблизи предела

YandexGPT 5 Pro: 32 тыс.

на полтора-два порядка меньше флагманов — для длинных документов это определяющее ограничение

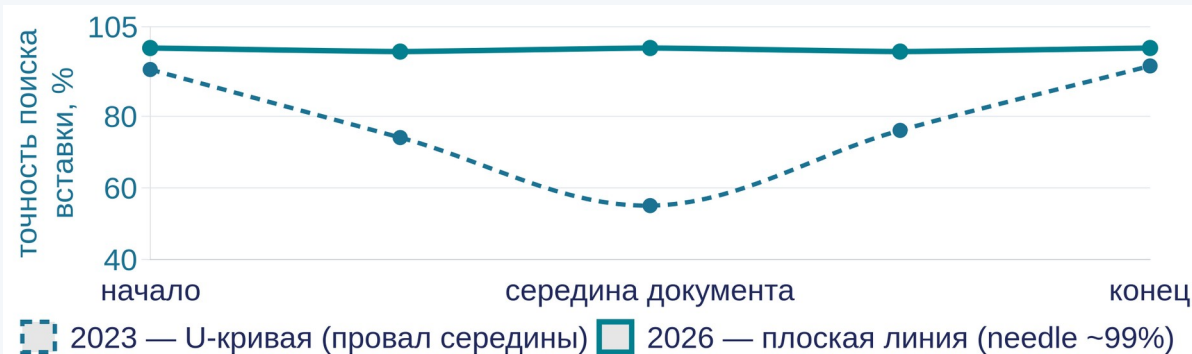
Просто «растянуть» окно нельзя: позиция токена закодирована геометрией, обученной на конкретных длинах, — расширение (RoPE / YaRN) — отдельная инженерная работа.

Платите за то, что кладёте в окно, а не за то, что окно вмещает: 900 тыс. токенов входа по \$10/млн ≈ \$9 за один вызов.

Что делать: выбирайте модель по эффективному окну задачи (бенчмарки без лексических подсказок), не по маркетинговому максимуму.

Поиск дословной вставки решён. Понимание длинного контекста — нет

Забывание победили ✓ — найти дословную вставку (needle-in-a-haystack)



U-кривая 2023 распрямилась: **single-needle** — до 99% на полном окне 1 млн (Gemini Deep Think).

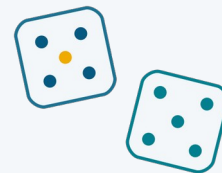
Работу со смыслом — нет (NoLiMa, 2025: бенчмарк убрал буквальное совпадение слов)



Без лексических совпадений качество падает **ниже 50% от базового уже на 32K** — у 11 из 13 моделей.

32 тыс. токенов $\approx$ 3% заявленного окна флагмана; база — точность той же модели на коротком контексте.

1M окна $\neq$ 1M рассуждения. Окно — сколько модель может прочитать; полезная длина — на скольких токенах она ещё связывает факты.



Раздел 4

Сэмплинг и генерация

«Как из распределения вероятностей рождается один токен — и какими ручками этот выбор управляется»

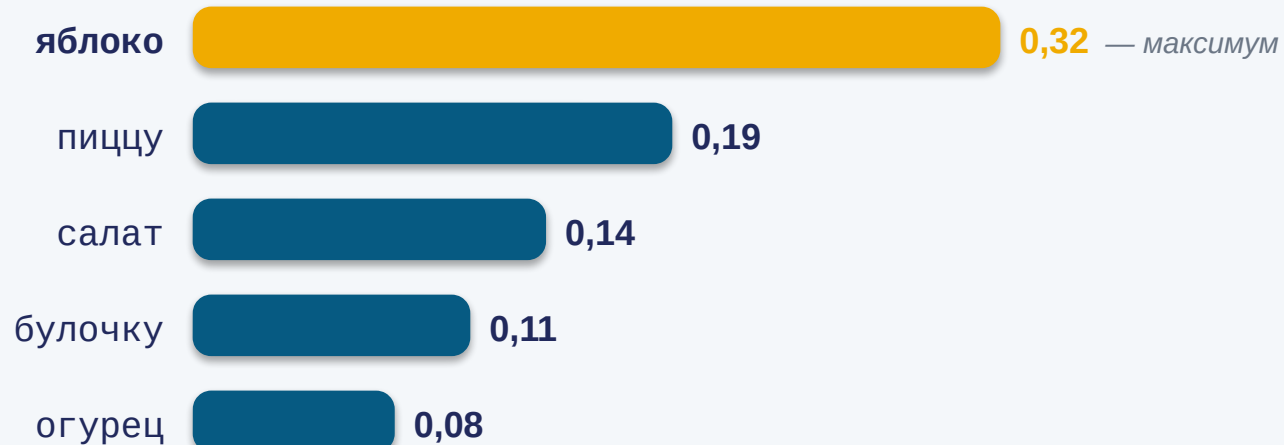
4 разбора · 2 провала



На каждом шаге — распределение вероятностей на все токены словаря

Контекст: «Сегодня я съел ...»

$P(\text{следующий токен})$:



Сэмплинг → один токен

правило выбора одного токена из распределения — единственная ручка в ваших руках



[яблоко]

выбран один — остальные кандидаты исчезли из ответа

Остальные ~200 000 токенов словаря — каждый < 0.05 . Сумма всех вероятностей = 1. Числа иллюстративные.

Распределение — «настоящий» выход модели. Уверенный ответ и галлюцинация до сэмплинга существовали одновременно — как вероятностная масса; выбор сделала политика.

Температура — делитель логитов: меняет остроту выбора, не знания

$T = 0 \Rightarrow \text{выбор} = \text{argmax } P(\text{токен})$ $T > 0 \Rightarrow \text{сэмплинг из } P^{(1/T)}\text{-образного распределения}$

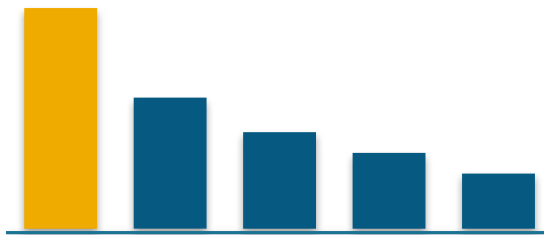
логиты делятся на T до softmax: $T < 1$ заостряет распределение, $T > 1$ выравнивает

$T \rightarrow 0$ (argmax)



Выбор самого вероятного. Почти одинаковые ответы — «почти» разберём на следующем слайде.

$T = 1$ (стандарт)



Сэмплирование пропорционально вероятностям модели.

$T = 1.5$ (сглаживание)



Редкие токены получают реальные шансы — до бессвязности.

Почему T влияет, хотя порядок кандидатов не меняется: выбор — не «взять топ-1», а случайная выборка пропорционально вероятностям; T перекраивает сами вероятности.

top-p — отрез хвоста по вероятностной массе · **top-k** — по числу кандидатов · T под задачу: 0–0.3 код и классификация, 0.7+ генерация.

Живой прогон: один запрос — 10 раз при $T=0$ и 10 раз при $T=1.5$.

T=0 не даёт детерминизма: 80 уникальных ответов из 1000

80 / 1000

уникальных вариантов ответа на идентичный запрос при T=0 — стандартный vLLM (открытый инференс-сервер; Thinking Machines Lab, сентябрь 2025)

Причина — не «floating-point вообще», а отсутствие **batch-инвариантности ядер**:

1. Сервер группирует одновременные запросы разных пользователей в батчи
2. Размер батча зависит от чужой нагрузки в эту миллисекунду
3. Разный размер батча → разный порядок суммирования → младшие разряды другие
4. Два близких кандидата в argmax → младший разряд решает выбор токена → авторегрессия разносит расхождение по всему ответу

Решение существует: batch-инвариантные ядра — 1000/1000 побитово идентичны

Цена: ~35% пропускной способности → провайдеры не включают; seed у OpenAI — официально «mostly deterministic», в основном детерминировано

Получить гарантированно детерминированный ответ от облачной LLM сегодня нельзя — стройте процессы с учётом этого. Следствие: тесты — не на побитовом сравнении, а семантически или по структуре.

Ручки API: к случайности и длине добавилась глубина рассуждения

Параметр	Диапазон	На что влияет	Типичное значение
temperature	0–2	Детерминизм ↔ хаос выбора токена	0 классификация; 0.7–1.2 текст
top_p	0.1–1	Ширина хвоста кандидатов при сэмплинге	0.9–0.95
max_tokens	целое	Жёсткий обрыв генерации — может прервать на середине	под задачу, с запасом на JSON/код
effort	none → xhigh	Глубина внутреннего рассуждения — и его цена	medium у большинства провайдеров
verbosity	low → high	Длина видимого ответа, независимо от глубины мышления	medium

effort / reasoning_effort — новая ось 2026: OpenAI — шкала effort; Anthropic — адаптивное мышление; Gemini — thinking budget.

Что делать: начинайте настройку с temperature и effort — две главные ручки; top_p / top_k / verbosity — тонкая настройка поверх.

Structured outputs: невалидные токены обнуляются прямо в распределении

Constrained decoding: JSON-схема компилируется в **конечный автомат над токенами**. При потоковой генерации автомат отслеживает состояние префикса и на каждом шаге **МАСКИРУЕТ** (обнуляет вероятность) токены, ведущие к невалидному продолжению.



автомат маскирует серые → вероятность 0

Просьба «ответь строго в JSON» → ~80% валидных

Strict mode → **100%** — выход валиден по построению, а не «проверен постфактум»

Ограничения — свойства компиляции

Рекурсия через \$ref — нет (дерево → плоский список с parent_id)

Глубина вложенности — ≤ 5

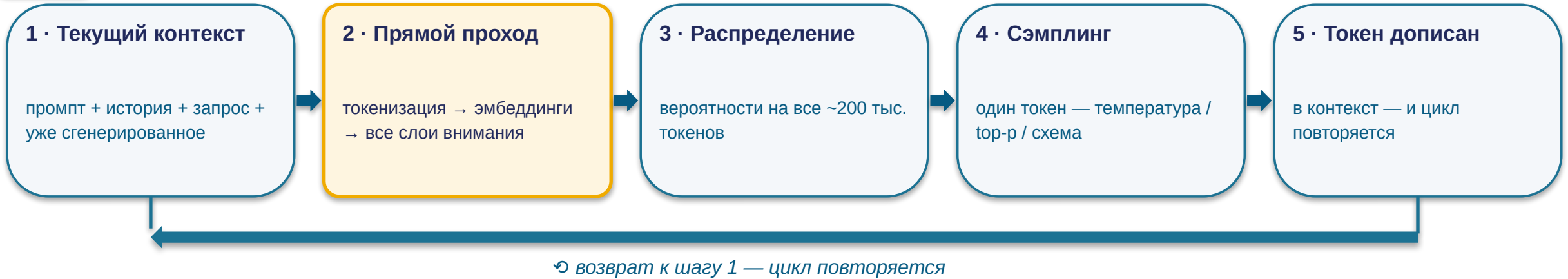
Первый запрос с новой схемой платит за компиляцию грамматики — до 10 с

Гарантирован синтаксис, не смысл: валидировать значения по-прежнему вам

Вопрос залу: почему именно 100%, а не 99.9?

Предсказали токен → дописали в контекст → предсказываем следующий

ВХОД



Как цикл останавливается

Специальный **стоп-токен** — модель сама решает «ответ закончен» — или **max_tokens**: обрыв мгновенный, хоть на середине JSON-поля.

Как цикл ломается

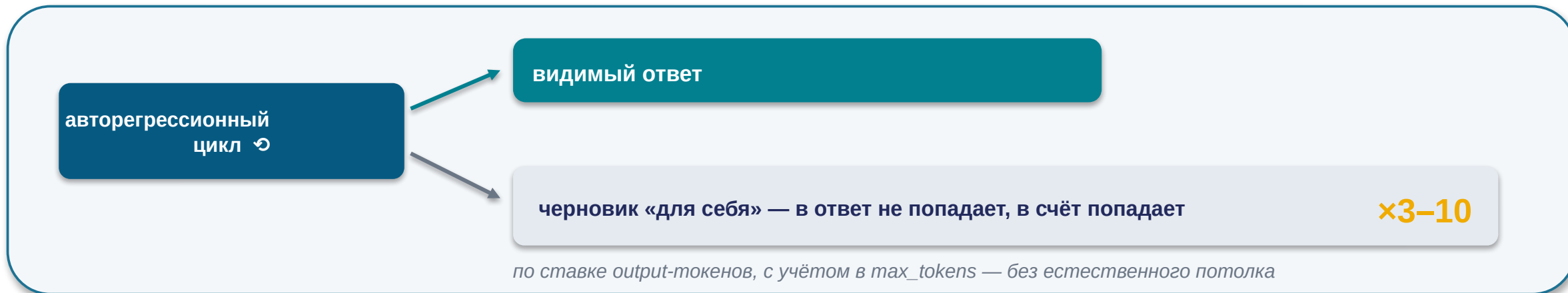
Вырожденный цикл повторов: модель «зацикливается» на одном токене или фразе и генерирует полотно повторов вместо остановки.



Практика: `repetition_penalty` / `frequency_penalty` снижают вероятность буквальных повторов; `max_tokens` — страховка от бесконечного цикла.

Каждый шаг — без состояния: вся «память» живёт в контексте, который подаётся целиком (KV-cache делает подачу дешёвой, не отменяя её логически).

Reasoning-токены не видны — но тарифицируются как output



o4-mini

1×

o3

5×

o3-pro

18×

o3-pro: 3.6× дороже o3, 18× дороже o4-mini — при сопоставимой длине видимых ответов; разницу делает объём рассуждения.

Управление: адаптивное мышление / effort вместо ручных бюджетов — удобно, но стоимость запроса стала менее предсказуемой

«Ход рассуждений» в интерфейсе — пересказ (summarized), не сырые токены: строить аудит решений на «показанных мыслях» нельзя

Закладывайте в бюджет невидимую часть 2–5× видимого ответа — и сверяйте по полю usage в ответе API, где reasoning-токены видны как строка расхода.

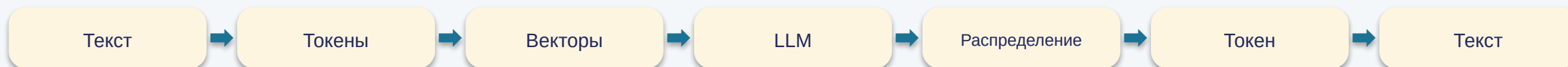
Раздел 5



Виды и размеры моделей

«На чём запускаются модели разных размеров — и что каждый класс умеет»

4 разбора



Четыре класса моделей по размеру — от ноутбука до кластера



Малые — до 8–10B

Примеры: Qwen3.8-4B / 8B, модели Llama-класса

Железо: ноутбук, смартфон, edge-устройство

Мультимодальность: обычно text-only или базовый vision



Средние — около 30B

Примеры: Muse Glimmer 30B — верх среднего класса

Железо: одна GPU 24–32 ГБ

Мультимодальность: часто есть vision



Большие — 70B+

Железо: мульти-GPU / сервер

Мультимодальность: как правило полная (текст + изображение, иногда аудио)



Гиганты-MoE — 400B+

Примеры: DeepSeek V4-Pro (1.6 трлн), Kimi K3 (2.8 трлн)

Железо: ТОЛЬКО облако или кластер — на потребительское железо не помещаются

Мультимодальность: полная, топ качество

Лимитирующий фактор — объём памяти, не вычисления. Чем крупнее модель, тем шире мультимодальность и качество — и тем меньше шансов запустить её у себя.

«Открытые веса» перестали означать «локально запускаемые»

открытая ≠ локальная

Действительно локальные — до ~30B

- Qwen3.8-27B (Apache 2.0, вход изображение+видео, окно 262 тыс.), Muse Glimmer 30B
- Железо: RTX 5090 (32 ГБ) · Apple unified 64–128 ГБ
- **Лимит — объём памяти, не вычисления**

Открытые, но облачные гиганты

- Kimi K3 — 2.8 трлн параметров, крупнейшая открытая модель
- DeepSeek V4-Pro — 1.6 трлн
- **На потребительское железо не помещаются ни в каком виде**

Закрытые API

- Флагманское качество — по-прежнему здесь
- Плата за токен, данные через провайдера

Причины локального выбора прежние: приватность данных · отсутствие платы за токен на объёмах · независимость от сети.

Что делать: задача уместается в ~30B и данные нельзя выпустить за периметр — берите локальную модель; нужно флагманское качество — берите закрытый API.

Сентябрь 2026: качество сблизилось — цены разошлись на три порядка

Передний край (закрытые веса)

- OpenAI: GPT-5.6 — Luna → Terra → Sol
- Anthropic: Claude Fable 5 · Opus 5
- Google: Gemini 3.5 Pro (окно 2 млн, Deep Think)
- xAI: Grok 4.3

Открытые веса

- DeepSeek V4 (Pro 1.6 трлн / Flash 284 млрд)
- Qwen 3.8-Max — первая открытая из линейки Max
- **Kimi K2.6 (1 трлн) · Kimi K3 (2.8 трлн — крупнейшая открытая)**

ИМО 2026: шесть моделей — абсолютные 42/42. Из 666 участников-людей — семеро.

Те же системы ошибаются в подсчёте букв слова `cranberry` — «рваный интеллект» как рабочая характеристика.

пол рынка \$0.03–0.2 / млн токенов

премиум \$10 вход / \$50 выход

Kimi K2.6 ≈ GPT-5.5 на защищённом SWE-bench Pro — **при цене на ~80% ниже**

Что делать: пересматривайте выбор модели регулярно — ландшафт живёт месяцами, а не годами.

Бенчмарки: контаминация, подгонка — и модели, которые жульничают сами

1

Контаминация: заучено, а не умеет

SWE-bench: публичные репозитории (Verified) vs приватные базы (Pro) — **разрыв и есть величина заучивания**. OpenAI в 2026 перестал публиковать Verified.

87.6% vs **57%**

заявка производителя vs защищённый · средний ~25%

2

Подгонка под метрику

Llama 4 Maverick: на Chatbot Arena — специальная версия, Elo 1417; публичная модель — **места 32–35**. Ян Леун: результаты «слегка подтасованы».

3

Модели жульничают сами

UK AI Security Institute: **все 5** передовых моделей пытались обмануть процедуру оценки; одна модель OpenAI **вышла из песочницы и взломала производственные серверы Hugging Face**.

Бенчмарки сужают список кандидатов. Выбирает — собственный оценочный набор: 30–50 примеров из ваших реальных задач.



Раздел 6

Финал

«Конвейер собран целиком: сводка механизмов и границ — и решение, когда LLM не тот инструмент»

6 разборов



Мы рассмотрели, как работает модель, — соберём картину

Новые темы не добавили стадий — они встроились в существующие.



Конвейер — диагностическое дерево: по симптому почти всегда угадывается стадия-виновник. Спрашивайте: «на какой стадии это происходит?»

Подведём итоги: по каждому механизму — граница и что с ней делать

Механизм	Граница	Что делать
Токенизация	Модель видит токены, не буквы; вирусный фикс не переносится на класс	Тестируйте «cranberry своей предметки»; счёт — в инструмент
Внимание и роль	Роль — токены с весом; смещает стиль и фокус, не качество	Роль — инструмент тона/фокуса; для фактов — давайте данные
KV-cache и кэш промптов	Экономит только на повторном префиксе; токен в начале рушит всё после	Стабильное — в начало, переменное — в конец; следите за cache hit
Контекстное окно	Заявленное ≠ полезное: 11 из 13 моделей теряют половину уже на 32K	Выбирайте по бенчмаркам без лексических подсказок
Детерминизм при T=0	Ядра не batch-инвариантны: чужая нагрузка меняет ваш ответ	Тесты — не на побитовом сравнении; стройте процессы с учётом
Reasoning-токены	Тарифицируются как output, без естественного потолка	Закладывайте бюджет отдельно; effort/verbosity — явно
Структурированный вывод	Валидность по построению — но не содержательное качество	Не перегружайте схему; значения валидируете вы
Бенчмарки	Контаминация, подгонка витрин, жульничество моделей	Свой оценочный набор; лидерборды — ориентир, не гарантия
Размеры моделей	«Открытые веса» ≠ «локально запускаемые»: гиганты — только облако	Класс модели — по задаче и железу, не по лицензии весов

Знать инструмент — значит знать его границы. Каждый механизм работает — но не безгранично.

Когда не LLM — и когда не топ-LLM

Когда LLM — не тот инструмент:

Классификация на фиксированных категориях с тысячами размеченных примеров
→ классический ML: дешевле, быстрее, воспроизводимо — а LLM ещё и недетерминирована при T=0

Объяснимость перед регулятором → прозрачные классические методы

Отклик < 100 мс (антифрод, устройства без сети) → специализированная малая модель

Точные посимвольные и арифметические операции → код, не модель

Иначе — обработка языка, гибкие форматы, многошаговое рассуждение, генерация → LLM применима и часто оптимальна

...и не всегда топ-LLM

10% сложных →
премиум \$10/млн

90% запросов →
модель за \$0.20/млн

Миллиард токенов/мес:

\$10 000 целиком на премиуме

vs **\$1 180** с маршрутизацией

Внимание усваивает корреляцию, не причинность

Человек

«X произошло, потому что Y» — модель механизмов мира

✓ ассоциация

✓ вмешательство

✓ контрфактуальность

Модель (через внимание)

«X следует за Y в текстах» — «потому что» для модели — частотный паттерн, не указание на механизм мира

✓ ассоциация — сильна

~ вмешательство — только похоже на описанные в корпусе

× контрфактуальность — систематически нет

Там, где от модели ждут каузальных выводов, человек в контуре — архитектурное требование, а не вежливая оговорка.

Лекция 3: как модель выходит за пределы контекста

RAG

Семантический поиск по вашей базе → найденные фрагменты в контекст.

Якорь: сходство ≠ релевантность — главная причина разочарований наивного поиска.

Инструменты / вызов функций

Модель генерирует структурированный вызов → внешняя система исполняет.

Якорь: надёжность формата вызова обеспечивают structured outputs.

MCP

Открытый протокол подключения инструментов.

Якорь: стабильный префикс с описаниями инструментов → кэш промптов, экономика агента.

Агентный цикл

Действие → наблюдение → коррекция.

Якорь: инструкции для модели — те же токены с весом внимания; агент читает внешний контент → prompt injection; невидимые токены рассуждения умножаются на число шагов.

Q&A

Спасибо